FINÁLNÍ PROJEKT
č. 2

engeto

Autor: Pavel Illek
Datum: 08.11. 2026

OBSAH

ZADÁNÍ **3**
Účel dokumentu 3
Přístupové údaje: 3
Omezení 3
Prerekvizity 3
Poznámky 4
TESTOVACÍ SCÉNÁŘE **4**
Testovací okruh 1: Autentizace uživatele 4
Testovací okruh 2: Metoda POST 10
Testovací okruh 3: Metoda GET 16
Testovací okruh 3: Metoda PUT 18
Testovací okruh 4: Metoda DELETE 22
EXEKUCE TESTŮ **24**
Výsledky okruhu 1: Autentizace uživatele 24
Výsledky okruhu 2: Metoda POST 25
Výsledky okruhu 3: Metoda GET 25
Výsledky okruhu 4: Metoda PUT 26
Výsledky okruhu 5: Metoda DELETE 26
BUG REPORT **27**
ZÁVĚR **28**

ZADÁNÍ

Účel dokumentu

Cílem finálního projektu je otestovat funkčnost aplikace, která slouží k manipulaci s daty o studentech. Aplikace má rozhraní REST-API, které umožňuje vytvoření, smazání a získání dat. V dokumentu jsou zaznamenány postupy testování a testovací výsledky. Tento dokument má především dokázat znalost práce s aplikací Postman, DBeaver, znalost základních SQL a HTTP požadavků.

Přístupové údaje:

Databáze	**Default scheme**: students **Host**: test-app.engeto.cz **Port**: 5433
REST-API REST-API dokumentace	https://test-app.engeto.cz/students https://test-app.engeto.cz/docs

Omezení

Testování se zaměřuje čistě na CRUD funkcionalitu endpointů zveřejněných v REST-API dokumentaci, tedy konkrétně na metody GET, POST, PUT a DELETE. Nebyly provedeny bezpečnostní, zátěžové, výkonnostní ani jiné testy. V dokumentu jsou popsány metody testování a data použitá v testech.

Prerekvizity

Postman	Nástroj pro testování API.
Dbeaver	DBeaver je SQL klientský software a nástroj pro správu databází.

Přístupové údaje k API. Tyto údaje nejsou součástí dokumentu jelikož se jedná o citlivé údaje sdílené zabezpečenou cestou.

Poznámky

Nezapomeňte, že v IT se data musí někde uložit a poté získat. Proto ověřte, že data jsou správně uložena a získávána z databáze.
Nezapomeňte do testovacích scénářů uvést testovací data, očekávaný výsledek včetně těla odpovědi a stavových kódů.

V dokumentu se objevuje pojem *platné_heslo*. Jedná se o platné přihlašovací heslo do databáze DBeaver sdílené bezpečnou cestou, které není možné zveřejnit v tomto dokumentu.

TESTOVACÍ SCÉNÁŘE

Na základě uvedených testovacích scénářů jsem ověřil funkčnost aplikace.

Testovací okruh 1: Autentizace uživatele

Okruh zaměřený na autentizaci uživatele. Jedná se o odesílání požadavků na endpoint https://test-app.engeto.cz/login.

Seznam testovacích scénářů

ID	Název testu
TC_AUTH_01	Autentizace uživatele bez platného hesla
TC_AUTH_02	Odeslání požadavku pro přihlášení s nesprávným datovým typem v těle požadavku
TC_AUTH_03	Odeslání požadavku pro přihlášení s neplatnými přihlašovacími údaji
TC_AUTH_04	Odeslání požadavku GET bez bearer tokenu
TC_AUTH_05	Získání bearer tokenu s validními údaji

TC_AUTH_01: Autentizace uživatele bez platného hesla	
Popis	Ověření, že API neposkytne bearer token bez zadání platného hesla.
Počáteční podmínky	Uživatel má nainstalovaný postman.

Testovací data	{
	"login": "testing_academy_2026_03_26_3",
	"password": ""
	}
	
	https://test-app.engeto.cz/login

Hlavička requestu	Content-type: application/json

Tělo requestu	{
	"login": "testing_academy_2026_03_26_3",
	"password": ""
	}

SQL query pro ověření API požadavku	N/A
Kroky testu	1. Uživatel vyplní v aplikaci Postman uvedený endpoint a vyplní JSON tělo požadavku. 2. Uživatel odešle POST požadavek s vyplněnými hodnotami. 3. Uživatel zkontroluje tělo odpovědi a kód odpovědi
Očekávaný výsledek	- HTTP kód odpovědi 422 Payload failed validation. - tělo odpovědi obsahuje validační chybu

TC_AUTH_02: Odeslání požadavku pro přihlášení s nesprávným datovým typem v těle požadavku	
Popis	Ověření, zda API správně zpracuje požadavek s neplatným datovým typem v těle požadavku.
Počáteční podmínky	Uživatel má nainstalovaný postman.

Testovací data	{
	"login": testing_academy_2026_03_26_3,
	"password": "*platné_heslo*"
	}
	
	https://test-app.engeto.cz/login

Hlavička requestu	Content-type: application/json

Tělo requestu	{
	"login": testing_academy_2026_03_26_3,
	"password": "platné_heslo"
	}

SQL query pro ověření API požadavku	N/A
Kroky testu	1. Uživatel vyplní v aplikaci Postman uvedený endpoint a vyplní JSON tělo požadavku. 2. Uživatel odešle POST požadavek s vyplněnými hodnotami. 3. Uživatel zkontroluje tělo odpovědi a kód odpovědi
Očekávaný výsledek	- HTTP kód odpovědi 422 Payload failed validation. - tělo odpovědi obsahuje validační chybu

TC_AUTH_03: Odeslání požadavku pro přihlášení s neplatnými přihlašovacími údaji	
Popis	Ověření odpovědi na požadavek s neplatnými přihlašovacími údaji.
Počáteční podmínky	Uživatel má nainstalovaný Postman.

Testovací data	{
	"login": "login_login",
	"password": "heslo_heslo"
	}
	
	https://test-app.engeto.cz/login

Hlavička requestu	Content-type: application/json

Tělo requestu	{
	"login": "login_login",
	"password": "heslo_heslo"
	}

SQL query pro ověření API požadavku	N/A
Kroky testu	1. Vyplň v aplikaci Postman uvedený endpoint a vyplní JSON tělo požadavku. 2. Odešli POST požadavek s vyplněnými hodnotami. 3. Zkontroluj tělo odpovědi a kód odpovědi
Očekávaný výsledek	- HTTP kód odpovědi 401 Unauthorised - Tělo odpovědi obsahuje informaci o “invalid login or password.”

TC_AUTH_04: Odeslání požadavku GET bez bearer tokenu	
Popis	Ověření, že odpověď API nevrátí informace z databáze bez platného bearer tokenu.
Počáteční podmínky	Existuje alespoň jeden student v databázi *students*. Uživatel má nainstalovaný Postman a DBeaver.
Testovací data	ID existujícího studenta v databázi: 4 https://test-app.engeto.cz/students/4
Hlavička requestu	N/A
Tělo requestu	N/A
SQL query pro ověření API požadavku	SELECT * FROM students WHERE ID = 4
Kroky testu	1. Zkontroluj v DBeaver, zda se student s ID 4 nachází v databázi 2. V aplikaci Postman odešli požadavek GET https://test-app.engeto.cz/students/4. 3. Ověř API odpověď a response code v těle odpovědi.
Očekávaný výsledek	- HTTP kód odpovědi 401 Unauthorised - Tělo odpovědi obsahuje informaci, že je bearer token neplatný nebo chybí - Student s ID 4 se skutečně v databázi nachází

TC_AUTH_05: Získání bearer tokenu s validními údaji	
Popis	Získání validního bearer tokenu pro odesílání požadavků
Počáteční podmínky	Uživatel má nainstalovaný postman. Uživatel má platné přihlašovací údaje.

Testovací data	{
	"login": "<login>",
	"password": "<heslo>"
	}
	
	https://test-app.engeto.cz/login

Hlavička requestu	Content-Type: application/json

Tělo requestu	{
	"login": "<login>",
	"password": "<heslo>"
	}

SQL query pro ověření API požadavku	N/A
Kroky testu	1. Odešli požadavek s platnými přihlašovacími údaji na POST https://test-app.engeto.cz/login 2. Zkontroluj tělo odpovědi
Očekávaný výsledek	- Kód odpovědi je 200 - bearer token se nachází v těle odpovědi

Testovací okruh 2: Metoda POST

Okruh zaměřený na vytváření nových záznamů v databázi. Jedná se o odesílání požadavků POST na endpoint https://test-app.engeto.cz/students.

Seznam testovacích scénářů

ID	Název testu
TC_POST_01	Vytvoření nového záznamu studenta
TC_POST_02	Vytvoření záznamu studenta s již existující e-mailovou adresou v databázi
TC_POST_03	Validace nesprávného formátu e-mailové adresy
TC_POST_04	Odeslání požadavku s prázdným tělem požadavku
TC_POST_05	Neplatný klíč v těle požadavku

TC_POST_01: Vytvoření nového záznamu studenta	
Popis	Ověření, že lze vytvořit záznam studenta v databázi.
Počáteční podmínky	Nainstalovaný Postman a DBeaver. Uživatel má platný bearer token.
Testovací data	Totožná s tělem requestu. Platný bearer token.
Hlavička requestu	Content-Type: application/json Authorization: Bearer <token>

Tělo requestu	{
	"firstName": "Tester",
	"lastName": "Tester",
	"email": "test@example.cz",
	"isEuCitizen": true,
	"zipCode": 11100,
	"numberOfFailedStudies": 0
	}

SQL query pro ověření API požadavku	SELECT * FROM students WHERE email = 'test@example.cz';
Kroky testu	1. Odešli požadavek POST s uvedeným tělem požadavku na endpoint https://test-app.engeto.cz/students. 2. Zkontroluj HTTP status odpovědi a tělo odpovědi. 3. Zkontroluj záznam v databázi pomocí SQL query, zda odpovídá datům odpovědi v Postmanovi.
Očekávaný výsledek	- HTTP response code je 201 - Tělo odpovědi obsahuje zadaná data v těle požadavku včetně unikátního ID - SQL dotaz zobrazí záznam uživatele v databázi - Záznam v databázi obsahuje data zadaná v těle požadavku.

Očekávaná struktura JSON odpovědi	{
	"id": <id_studenta>,
	"firstName": "Tester",
	"lastName": "Tester",
	"email": "test@example.cz",
	"isEuCitizen": true,
	"zipCode": "11100",
	"numberOfFailedStudies": 0
	}

TC_POST_02: Vytvoření záznamu studenta s již existující e-mailovou adresou v databázi	
Popis	Ověření odpovědi API požadavku POST. Podle dostupné dokumentace nelze vytvořit v databázi záznam studenta s duplicitní e-mailovou adresou.
Počáteční podmínky	Nainstalovaný Postman a DBeaver. Uživatel má platný bearer token. V databázi existuje alespoň jeden záznam studenta s e-mailovou adresou.
Testovací data	Totožná s tělem requestu. Platný bearer token.
Hlavička requestu	Content-Type: application/json Authorization: Bearer <token>

Tělo requestu	{
	"firstName": "Petr",
	"lastName": "Tester",
	"email": "test@example.cz",
	"isEuCitizen": true,
	"zipCode": null,
	"numberOfFailedStudies": 0
	}

SQL query pro ověření API požadavku	SELECT * FROM students WHERE email = 'test@example.cz';
Kroky testu	4. Odešli požadavek POST s uvedeným tělem požadavku na endpoint https://test-app.engeto.cz/students. 5. Zkontroluj HTTP status odpovědi a tělo odpovědi. 6. Zkontroluj záznam v databázi pomocí SQL query, zda odpovídá datům odpovědi v Postmanovi.
Očekávaný výsledek	- HTTP response code je 400. - Tělo odpovědi obsahuje informaci, že student s danou e-mailovou adresou již existuje. - SQL dotaz zobrazí pouze jeden záznam uživatele v databázi s e-mailovou adresou.

TC_POST_03: Validace nesprávného formátu e-mailové adresy	
Popis	Ověření, jestli dojde k uložení e-mailové adresy do databáze v nesprávném formátu.
Počáteční podmínky	Nainstalovaný Postman a DBeaver. Uživatel má platný bearer token. V databázi existuje alespoň jeden záznam studenta s e-mailovou adresou.
Testovací data	Totožná s tělem requestu. Platný bearer token.
Hlavička requestu	Content-Type: application/json Authorization: Bearer <token>

Tělo requestu	{
	"firstName": "Petr",
	"lastName": "Tester",
	"email": "invalidemail.com",
	"isEuCitizen": true,
	"zipCode": null,
	"numberOfFailedStudies": 0
	}

SQL query pro ověření API požadavku	SELECT * FROM students WHERE email = 'invalidemail.com';
Kroky testu	1. Odešli POST požadavek na https://test-app.engeto.cz/students s tělem requestu. 2. Zkontroluj odpověď požadavku. 3. Zkontroluj záznam v databázi pomocí SQL query.
Očekávaný výsledek	- HTTP response code je 422 Payload failed validation. - Tělo odpovědi obsahuje informaci, že v těle požadavku se nachází nevalidní informace. - SQL dotaz nenalezne uživatele s e-mailovou adresou invalidemail.com.

TC_POST_04: Odeslání požadavku s prázdným tělem požadavku	
Popis	Ověření, že nedojde k uložení studenta do databáze s prázdnými hodnotami.
Počáteční podmínky	Nainstalovaný Postman a DBeaver. Uživatel má platný bearer token.
Testovací data	Platný bearer token.
Hlavička requestu	Content-Type: application/json Authorization: Bearer <token>
Tělo requestu	{}
SQL query pro ověření API požadavku	SELECT COUNT(*) FROM students;
Kroky testu	1. Zkontroluj počet studentů v databázi pomocí SQL query 2. odešli POST požadavek na endpoint https://test-app.engeto.cz/students s prázdným tělem požadavku. 3. Zkontroluj tělo odpovědi. 4. Zkontorluj počet záznamů v databázi.
Očekávaný výsledek	- HTTP odpověď požadavku je 422. - Nedojde k uložení záznamu s prázdnými hodnotami. - Tělo odpovědi obsahuje informaci o nesprávném požadavku. - Číslo počtu záznamů v databázi zůstalo nezměněno.

TC_POST_05: Neplatný klíč v těle požadavku	
Popis	Ověření, že API při odeslání požadavku s neplatným klíčem v těle požadavku zamítne požadavek o vytvoření studenta.
Počáteční podmínky	Nainstalovaný Postman a DBeaver. Uživatel má platný bearer token.
Testovací data	Totožná s tělem requestu. Platný bearer token.
Hlavička requestu	Content-Type: application/json Authorization: Bearer <token>

Tělo requestu	{
	"name": "Petr",
	"lastName": "Tester",
	"email": "invalid@test.com",
	"isEuCitizen": true,
	"zipCode": null,
	"numberOfFailedStudies": 0
	}

SQL query pro ověření API požadavku	SELECT * FROM students WHERE email = 'invalid@test.com';
Kroky testu	1. odešli POST požadavek na endpoint https://test-app.engeto.cz/students s vyplněným tělem požadavku. 2. Zkontroluj HTTP kód odpovědi a tělo odpovědi. 3. Zkontroluj databázi v DBeaveru
Očekávaný výsledek	- HTTP kód odpovědi 422 Payload failed validation. - Nedojde k uložení záznamu nového studenta do databáze. - V těle odpovědi API je zobrazená hláška o chybě

Testovací okruh 3: Metoda GET

Okruh zaměřený na získávání záznamů z databáze. Jedná se o odesílání požadavků GET na endpoint https://test-app.engeto.cz/students/{student_id}.

Seznam testovacích scénářů

ID	Název testu
TC_GET_01	Získání dat existujícího studenta
TC_GET_02	Získání dat neexistujícího studenta

TC_GET_01: Získání dat existujícího studenta	
Popis	Ověření správné odpovědi API na požadavek o data existujícího studenta.
Počáteční podmínky	Nainstalovaný Postman a DBeaver. Uživatel má platný bearer token. Existuje alespoň jeden student v databázi.
Testovací data	Platný bearer token. <ID studenta> z databáze. (Během testování platné ID studenta bylo 4)
Hlavička requestu	Authorization: Bearer <token>
Tělo requestu	N/A
SQL query pro ověření API požadavku	SELECT * FROM students WHERE id = 4;
Kroky testu	1. Odešli požadavek GET na endpoint https://test-app.engeto.cz/students/4 2. Zkontroluj HTTP kód odpovědi a tělo odpovědi. 3. Zkontroluj záznam v databázi pomocí SQL query a porovnej ho s odpovědí v Postmanovi.
Očekávaný výsledek	- HTTP kód odpovědi je 200 - tělo odpovědi obsahuje informace o studentovi s ID 4, které jsou stejné jako odpověď na SQL query

Očekávaná struktura JSON odpovědi	{
	"id": <id_studenta>,
	"firstName": "string",
	"lastName": "string",
	"email": "string",
	"isEuCitizen": true,
	"numberOfFailedStudies": 0
	}

TC_GET_02: Získání dat neexistujícího studenta	
Popis	Ověření správné odpovědi API na požadavek o data neexistujícího studenta.
Počáteční podmínky	Nainstalovaný Postman a DBeaver. Uživatel má platný bearer token.
Testovací data	<ID studenta> který se nenachází v databázi (v době psaní testu se v databázi nenacházel student s ID 600)
Hlavička requestu	Authorization: Bearer <token>
Tělo requestu	N/A
SQL query pro ověření API požadavku	SELECT * FROM students WHERE id = 600;
Kroky testu	1. Odešli požadavek GET na endpoint https://test-app.engeto.cz/students/600 2. Zkontroluj HTTP kód odpovědi a tělo odpovědi. 3. Zkontroluj záznam v databázi pomocí SQL query a porovnej ho s odpovědí v Postmanovi.
Očekávaný výsledek	- HTTP kód odpovědi 404 - informace v těle odpovědi, že student nemohl být nalezen - SQL query nenajde v databázi studenta s ID 600

Testovací okruh 3: Metoda PUT

Okruh zaměřený na upravování existujících záznamů v databázi. Jedná se o odesílání požadavků PUT na endpoint https://test-app.engeto.cz/students/{student_id}.

TC_PUT_01: Úprava dat studenta	
Popis	Ověření, že API požadavek upraví záznam existujícího studenta
Počáteční podmínky	Nainstalovaný Postman a DBeaver. Uživatel má platný bearer token. Existuje alespoň jeden student v testovací databázi.
Testovací data	ID studenta: 264
Hlavička requestu	Content-Type: application/json Authorization: Bearer <token>

Tělo requestu	{
	"firstName": "TesterTester",
	"lastName": "Tester2",
	"email": "test.test@example.com",
	"isEuCitizen": true,
	"zipCode": null,
	"numberOfFailedStudies": 0
	}

SQL query pro ověření API požadavku	SELECT * FROM students WHERE id = 264;
Kroky testu	1. Zkontroluj záznam v databázi pomocí SQL query. 2. Odešli PUT požadavek na endpoint https://test-app.engeto.cz/students/264. 3. Zkontroluj HTTP kód odpovědi a tělo odpovědi. 4. Zkontroluj záznam v databázi pomocí SQL query.
Očekávaný výsledek	- Kód odpovědi 200 a informace v odpovědi, že informace studenta byly aktualizované. - Tělo odpovědi obsahuje stejné informace jako odpověď SQL query po odeslání PUT požadavku.

TC_PUT_02: Duplicitní e-mailová adresa při úpravě záznamu studenta	
Popis	Ověření, že úpravou záznamu studenta nedojde k duplicitě e-mailové adresy v databázi.
Počáteční podmínky	Nainstalovaný Postman a DBeaver. Uživatel má platný bearer token. Existuje alespoň jeden student v testovací databázi.
Testovací data	<ID_STUDENTA>: ID existujícího studenta (264) <EXISTUJÍCÍ_EMAIL>: e-mail existujícího studenta s jinou hodnotou než student s ID 264 (test@example.cz)
Hlavička requestu	Content-Type: application/json Authorization: Bearer <token>

Tělo requestu	{
	"firstName": "TesterTester",
	"lastName": "Tester",
	"email": "test@example.cz",
	"isEuCitizen": true,
	"zipCode": null,
	"numberOfFailedStudies": 0
	}

SQL query pro ověření API požadavku	A) SELECT * FROM students WHERE id = 264; B) SELECT * FROM students WHERE email = 'test@example.cz';
Kroky testu	1. Zkontroluj záznam studenta s ID 264. (A) 2. Odešli požadavek PUT na endpoint https://test-app.engeto.cz/students/264 s tělem požadavku. 3. Zkontroluj HTTP kód odpovědi a tělo odpovědi. 4. Zkontroluj záznam studenta 264 v DBeaveru. (A) 5. Zkontroluj duplicitu e-mailu v databázi (B)
Očekávaný výsledek	- Kód odpovědi HTTP 400 bad request. - Tělo odpovědi obsahuje chybovou hlášku a popis chyby (e-mail v databázi již existuje). - Nedojde k úpravě hodnoty e-mailu v databázi. - V databázi se nachází pouze jeden záznam studenta s emailovou adresou test@example.cz.

TC_PUT_03: Prázdný požadavek v těle požadavku	
Popis	Ověření validace těla požadavku, pokud je tělo požadavku prázdné.
Počáteční podmínky	Nainstalovaný Postman a DBeaver. Uživatel má platný bearer token. Existuje alespoň jeden student v testovací databázi.
Testovací data	<ID_STUDENTA>: ID existujícího studenta (263) Platný bearer token
Hlavička requestu	Content-Type: application/json Authorization: Bearer <token>
Tělo requestu	{}
SQL query pro ověření API požadavku	SELECT * FROM students WHERE id = 263;
Kroky testu	1. Zkontroluj záznam studenta s ID 263. 2. Odešli požadavek PUT na endpoint https://test-app.engeto.cz/students/263 s tělem požadavku {}. 3. Zkontroluj HTTP kód odpovědi a tělo odpovědi. 4. Zkontroluj záznam studenta s ID 263 v DBeaveru.
Očekávaný výsledek	- Kód odpovědi HTTP 422 Unprocessable Entity. - Tělo odpovědi obsahuje chybovou hlášku a popis chyby (prázdné tělo požadavku). - Nedojde k úpravě hodnot v databázi.

TC_PUT_04: Odeslání PUT požadavku pouze s jedním atributem v těle požadavku	
Popis	Ověření, zda API umožňuje aktualizovat pouze jeden atribut existujícího studenta.
Počáteční podmínky	Nainstalovaný Postman a DBeaver. Uživatel má platný bearer token. Existuje alespoň jeden student v testovací databázi.
Testovací data	<ID_STUDENTA>: ID existujícího studenta (263) Platný bearer token
Hlavička requestu	Content-Type: application/json Authorization: Bearer <token>

Tělo requestu	{
	"lastName": "PetrPavel"
	}

SQL query pro ověření API požadavku	SELECT * FROM students WHERE id = 263;
Kroky testu	1. Zkontroluj záznam studenta s ID 263. 2. Odešli požadavek PUT na endpoint https://test-app.engeto.cz/students/263 s tělem požadavku. 3. Zkontroluj HTTP kód odpovědi a tělo odpovědi. 4. Zkontroluj záznam studenta s ID 263 v DBeaveru.
Očekávaný výsledek	- Kód odpovědi HTTP 200 OK. - Atribut lastName má hodnotu PetrPavel. - Ostatní atributy záznamu zůstávají beze změny.

Testovací okruh 4: Metoda DELETE

Okruh zaměřený na mazání existujících záznamů v databázi. Jedná se o odesílání požadavků DELETE na endpoint https://test-app.engeto.cz/students/{student_id}.

TC_DEL_01: Odstranění záznamu existujícího studenta	
Popis	Ověření, že dojde ke smazání záznamu studenta z databáze po odeslání validního požadavku DELETE.
Počáteční podmínky	V databázi existuje alespoň jeden záznam studenta. Platný bearer token.
Testovací data	<ID_STUDENTA>: ID existujícího studenta (264) Platný bearer token
Hlavička requestu	Authorization: Bearer <token>
Tělo requestu	N/A
SQL query pro ověření API požadavku	SELECT * FROM students WHERE id = '264';
Kroky testu	1. Zkontroluj záznam studenta v databázi pomocí SQL požadavku. 2. Odešli DELETE požadavek na endpoint https://test-app.engeto.cz/students/264. 3. Zkontroluj HTTP kód odpovědi a tělo odpovědi 4. Zkontroluj záznam studenta pomocí SQL požadavku, zda opravdu došlo k vymazání záznamu.
Očekávaný výsledek	- HTTP kód odpovědi 200 - Informace v těle odpovědi požadavku, že záznam byl úspěšně smazán - V databázi se daný záznam nenachází

TC_DEL_02: Odstranění záznamu neexistujícího studenta	
Popis	Ověření správné chybové hlášky v těle odpovědi po odeslání ID studenta, který se nenachází v databázi.
Počáteční podmínky	V databázi existuje alespoň jeden záznam studenta. Platný bearer token.
Testovací data	Platný bearer token
Hlavička requestu	Authorization: Bearer <token>
Tělo requestu	N/A
SQL query pro ověření API požadavku	SELECT * FROM students WHERE id = '700';
Kroky testu	1. Zkontroluj záznam studenta v databázi pomocí SQL požadavku, že skutečně neexistuje 2. Odešli DELETE požadavek na endpoint https://test-app.engeto.cz/students/700 3. Zkontroluj HTTP kód odpovědi a tělo odpovědi
Očekávaný výsledek	- HTTP kód odpovědi 404 - Informace v těle odpovědi požadavku, že studenta nelze najít - V databázi se skutečně žádný student s ID 700 nenachází

EXEKUCE TESTŮ

Testovací scénáře jsem provedl(a), přikládám výsledky testů.

Výsledky okruhu 1: Autentizace uživatele

TC ID	Skutečné chování	kód odpovědi	Stav	Poznámka
TC_AUTH_01	V těle odpovědi se nachází poznámka, že string “password” by měl obsahovat alespoň 6 znaků.	422	PASS	
TC_AUTH_02	V těle odpovědi se nachází informace o validation error.	422	PASS	
TC_AUTH_03	Tělo odpovědi hlásí chybu o neplatných přihlašovacích údajích (“Invalid credentials).	401	PASS	
TC_AUTH_04	V těle odpovědi se zobrazuje informace o nevalidním bearer tokenu. Žádná data z databáze nebyla zobrazena.	401	PASS	
TC_AUTH_05	V těle odpovědi se nachází bearer token i token type.	200	PASS	

Výsledky okruhu 2: Metoda POST

TC ID	Skutečné chování	kód odpovědi	Stav	Poznámka
TC_POST_01	Požadavek vytvořil nový záznam v databázi a vrátil očekávaný HTTP kód, ovšem došlo k převedení na velká písmena v příjmení studenta. Nedošlo k uložení prvního jména. Kvůli shodě ve jméně a příjmení. Nedošlo k uložení zip kódu. Ani tělo odpovědi požadavku neobsahuje atribut “zipCode”	201	FAIL	BUG_1, BUG_2, BUG_3
TC_POST_02	Odpověď požadavku správně vrátila informaci, že student s danou e-mailovou adresou již existuje.	400	PASS	
TC_POST_03	Požadavek vytvořil nový záznam studenta s neplatnou e-mailovou adresou. Nevalidní požadavek nebyl zamítnut.	201	FAIL	BUG_4
TC_POST_04	Došlo k uložení záznamu studenta s prázdnými (null) hodnotami. Tělo odpovědi neobsahuje chybovou hlášku.	201	FAIL	BUG_5
TC_POST_05	HTTP odpověď požadavku neodpovídá dokumentaci. Došlo k uložení nového záznamu do databáze. Klíč požadavku nebyl správně zvalidován.	201	FAIL	BUG_6

Výsledky okruhu 3: Metoda GET

TC ID	Skutečné chování	kód odpovědi	Stav	Poznámka
TC_GET_01	Tělo odpovědi obsahuje stejná data, jako odpověď SQL query v databázi.	200	PASS	
TC_GET_02	Tělo odpovědi obsahuje informaci, že studenta nelze nalézt. HTTP kód odpovědi je 500, což neodpovídá dokumentaci.	500	FAIL	BUG_7

Výsledky okruhu 4: Metoda PUT

TC ID	Skutečné chování	kód odpovědi	Stav	Poznámka
TC_PUT_01	Tělo odpovědi obsahuje stejné informace, jako informace v databázi. Aktualizace dat proběhla úspěšně. Číslo ale není povolený znak ve jméně, tedy nedochází ke správné validaci jména.	200	FAIL	BUG_8
TC_PUT_02	Tělo odpovědi obsahuje informace o úspěšném upravení dat studenta. Došlo k úpravě dat záznamu v databázi. Vznikl duplicitní záznam.	200	FAIL	BUG_9
TC_PUT_03	Tělo odpovědi obsahuje očekávané informace o chybějících informacích. Chybová hláška 422 odpovídá dokumentaci.	422	PASS	
TC_PUT_04	Došlo k úpravě pouze hodnoty atributu lastName. Ostatní atributy zůstaly beze změny.	200	PASS	Dokumentace API má jednoznačně uvádět, zda PUT podporuje částečnou aktualizaci. Pokud ji nepodporuje, je nutné ověřit a zdokumentovat odpovídající chování API.

Výsledky okruhu 5: Metoda DELETE

TC ID	Skutečné chování	kód odpovědi	Stav	Poznámka
TC_DEL_01	Kód odpovědi je 200 a informace v těle odpovědi udává, že student byl úspěšně smazán. Ovšem student z	200	FAIL	BUG_10

	databáze vymazán nebyl. SQL dotaz vrátil informace o studentovi.			
TC_DEL_02	Kód odpovědi je 500, ale dle dokumentace by měl být 404. Zároveň jazyk chyby je v češtině, kdežto ostatní informace v těle odpovědi byly v angličtině.	500	FAIL	BUG_11

BUG REPORT

Na základě provedených scénářů jsem objevil(a) uvedené chyby aplikace.

Bug	Priorita	Závažnost	Číslo testovacího scénáře	Mitigace
BUG_1: Atribut lastName převádí hodnotu automaticky na velká písmena.	Nízká	Nízká	TC_POST_01	Opravit ukládání hodnot do databáze tak, aby nedocházelo k automatickému převodu malých písmen na velká.
BUG_2: Pokud je první jméno (firstName) stejné jako příjmení (lastName), nedojde k uložení atributu “firstName”	Střední	Střední	TC_POST_01	Ošetřit případ, kdy je jméno stejné jako příjmení.
BUG_3: Zip kód se neukládá do databáze.	Vysoká	Kritická	TC_POST_01	Opravit ukládání hodnot do databáze.
BUG_4: Chybná validace hodnoty e-mailové adresy.	Vysoká	Kritická	TC_POST_03	Přidat validaci e-mailové adresy tak, aby ověřoval formát zadaného textu a případně vracel chybu 422, pokud je zadán nevalidní text.
BUG_5: Chybná validace prázdného těla požadavku.	Vysoká	Kritická	TC_POST_04	Upravit/přidat validaci těla požadavku. Nesmí dojít k uložení záznamu do databáze, pokud není vyplněno tělo požadavku dle dokumentace.
BUG_6: HTTP odpověď požadavku neodpovídá	Střední	Střední	TC_POST_05	Upravit validaci těla požadavku tak, aby validoval

dokumentaci. Klíč požadavku nebyl správně zvalidován.				hodnotu klíče v těle požadavku.
BUG_7: Chybný kód odpovědi.	Malá	Nízká	TC_GET_02	Opravit kód odpovědi, aby odpovídal dokumentaci.
BUG_8: Číslo není povolený znak ve jméně. Nesmí dojít k uložení takového jména.	Vysoká	Kritická	TC_PUT_01	Upravit validaci jména a příjmení tak, aby nemohlo dojít k uložení hodnoty obsahující číslo.
BUG_9: Chybná validace požadavku. Došlo k duplicitnímu záznamu v databázi.	Vysoká	Kritická	TC_PUT_02	Opravit validaci těla požadavku vůči databázi tak, aby nemohl vzniknout duplicitní záznam v DB.
BUG_10: Nedošlo k vymazání záznamu z databáze.	Vysoká	Kritická	TC_DEL_01	Upravit API tak, aby skutečně došlo k vymazání studenta z databáze.
BUG_11: Špatný jazyk v těle odpovědi - neodpovídá dokumentaci. Špatný HTTP kód odpovědi.	Střední	Střední	TC_DEL_02	Opravit jazyk těla odpovědi i kód odpovědi API.

ZÁVĚR

Tento dokument má především demonstrovat schopnost práce s aplikacemi Postman a DBeaver a také psaní testovacích scénářů i reportů těchto testů. Nejedná se o komplexní CRUD testování API a zápisů v databázi. Pokud by se jednalo o testování databáze v praxi, bylo by nutné vytvořit v aplikaci Postman testovací kolekce a skupinu datových sad. Nadále by bylo nutné rozšířit testovací scénáře tak, aby bylo pokryto co nejvíce proměnných (například hodnoty všech atributů, názvy všech atributů nebo kombinace zadávaných hodnot).

Pro druhý projekt Engeto testing academy bylo vytvořeno celkem 18 testovacích scénářů, které odhalily celkem 9 nálezů. Z těchto nálezů byly identifikovány 4 jako kritické, tedy takové chyby, které přímo ovlivňují správné chování databáze nebo API. Z tohoto důvodu lze konstatovat, že API neprošlo prvním kolem testování a před dalším kolem testů je nezbytné chyby opravit.